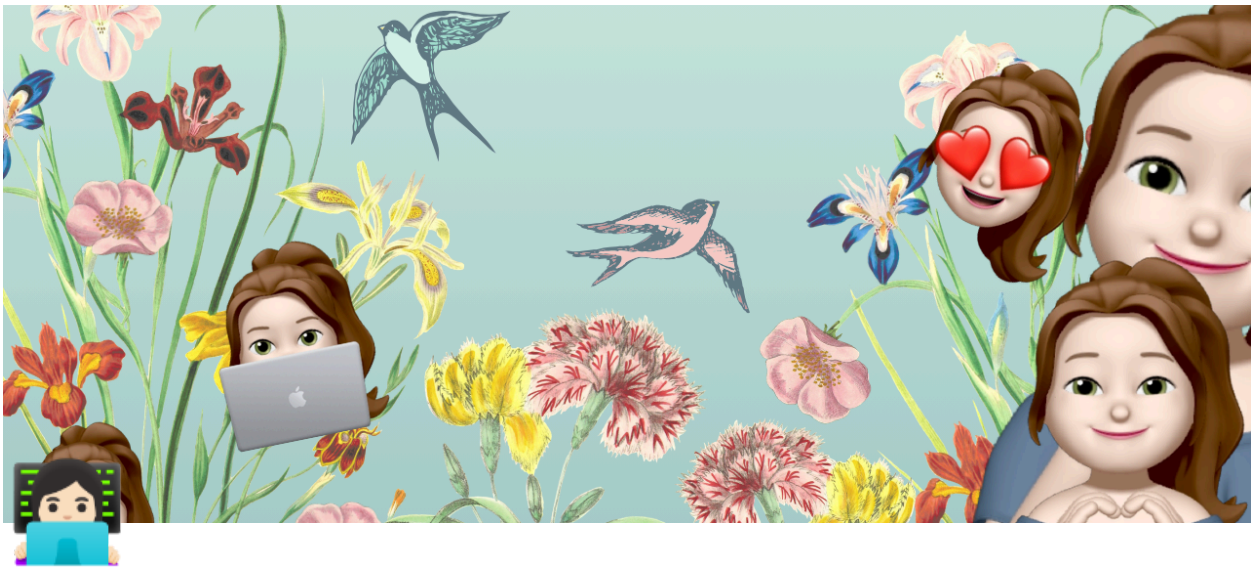


Documentação

Stock manager - Giulia Trevisan

Este projeto consiste em um sistema completo de **gestão de estoque**, desenvolvido como parte de um teste técnico com foco em **boas práticas de engenharia de software, organização de código, performance e arquitetura moderna**.

A aplicação foi projetada para simular um ambiente real, contemplando desde autenticação até gestão de dados e visualização de indicadores.

O sistema permite:

- Gerenciamento completo de produtos (CRUD)
- Controle de estoque com validações de negócio
- Gestão de usuários e níveis de acesso (admin/user)
- Visualização de dados através de dashboard com gráficos
- Exportação de relatórios (Excel e PDF)
- Autenticação segura utilizando JWT
- Paginação e filtros em listagens

- Execução completa via Docker (frontend + backend + banco)

Arquitetura e tecnologias

A aplicação segue uma arquitetura **fullstack desacoplada**, com comunicação via API REST.

Frontend

- **React 19** com TypeScript
- **Vite** como bundler
- **Material UI + TailwindCSS** para UI
- **Framer Motion** para animações
- **Axios** para consumo de API
- **React Router DOM** para navegação
- **Recharts** para gráficos
- **DataGrid (MUI)** para tabelas
- **js-cookie** para gerenciamento de autenticação
- **React Context API** para estado global (auth)

Backend

- **Node.js + Express**
- **Prisma ORM** para acesso ao banco
- **PostgreSQL**
- **JWT (jsonwebtoken)** para autentação
- **bcryptjs** para hash de senhas
- **Zod** para validação de dados
- **Swagger** para documentação de API
- **Seed automatizado** para dados iniciais

Infraestrutura

- **Docker + Docker Compose**
- Ambiente isolado com:
  - Backend
  - Frontend
  - Banco PostgreSQL
- Variáveis de ambiente via `.env`

Decisões técnicas

- Separação clara entre frontend e backend (arquitetura desacoplada)
- Uso de **Prisma** para garantir tipagem e produtividade no banco
- Validações implementadas tanto no backend (regras de negócio) quanto no frontend (UX)
- Uso de **Context API** para gerenciamento global de autenticação
- Aplicação de **useMemo e useCallback** para otimização de performance
- Uso de **interceptors do Axios** para tratamento global de autenticação (401)

Estrutura do projeto

O projeto está organizado em duas aplicações principais:

- **backend/** → API REST responsável pelas regras de negócio, autenticação e acesso ao banco
- **frontend/** → aplicação React responsável pela interface e consumo da API

Essa separação permite **escalabilidade, manutenção independente e melhor organização do código**.

Estrutura de diretórios

```
stock-manager/
│
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── seed.js
│   │   └── migrations/
│   │
│   ├── src/
│   │   ├── controllers/
│   │   │   ├── auth.controller.js
│   │   │   ├── product.controller.js
│   │   │   ├── user.controller.js
│   │   │   └── dashboard.controller.js
│   │   │
│   │   ├── services/
│   │   │   ├── auth.service.js
│   │   │   ├── product.service.js
│   │   │   ├── user.service.js
│   │   │   └── dashboard.service.js
│   │   │
│   │   ├── routes/
│   │   │   ├── auth.routes.js
│   │   │   ├── product.routes.js
│   │   │   ├── user.routes.js
│   │   │   └── dashboard.routes.js
│   │   │
│   │   ├── middlewares/
│   │   ├── validators/
│   │   ├── errors/
│   │   ├── lib/
│   │   ├── docs/
│   │   │   └── swagger.js
│   │   │
│   │   ├── app.js
│   │   └── server.js
│   │
│   └── Dockerfile
│
├── frontend/
│   ├── public/
│   │
│   ├── src/
│   │   ├── api/
│   │   ├── assets/
│   │   │
│   │   ├── components/
│   │   │   ├── auth/
```

```
│   │   │   ├── dashboard/
│   │   │   ├── inventory/
│   │   │   ├── users/
│   │   │   ├── layout/
│   │   │   ├── table/
│   │   │   ├── ui/
│   │   │   └── feedback/
│   │   │
│   │   ├── pages/
│   │   │   ├── auth/
│   │   │   ├── dashboard/
│   │   │   ├── inventory/
│   │   │   ├── users/
│   │   │   ├── profile/
│   │   │   └── faq/
│   │   │
│   │   ├── hooks/
│   │   ├── contexts/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── styles/
│   │   ├── types/
│   │   └── utils/
│   │
│   └── Dockerfile
│
├── docker-compose.yml
├── package.json
└── README.md
```

Organização do backend

O backend segue um padrão em camadas:

- **controllers/** → recebem a requisição HTTP e retornam resposta
- **services/** → concentram a lógica de negócio
- **routes/** → definem os endpoints da API
- **middlewares/** → autenticação, autorização e tratamento de erros
- **validators/** → validação de dados com regras de negócio
- **prisma/** → ORM e controle de migrations

Essa separação melhora:

- testabilidade
- manutenção
- reutilização de código

Organização do frontend

O frontend segue uma estrutura modular baseada em responsabilidade:

- **components/** → componentes reutilizáveis (UI, tabelas, modais)
- **pages/** → páginas da aplicação (rotas)
- **hooks/** → lógica reutilizável (ex: autenticação, formulários)
- **contexts/** → estado global (ex: AuthContext)
- **services/** → comunicação com API
- **routes/** → controle de rotas e proteção
- **utils/** → funções auxiliares
- **styles/** → temas e estilos globais

Destaque:

- Separação clara entre **UI, lógica e dados**
- Reaproveitamento de componentes
- Facilidade de escala

Execução do projeto

O projeto pode ser executado via Docker com um único comando:

```
docker compose up--build
```

Após a inicialização, os serviços estarão disponíveis em:

- Frontend: http://localhost:5173 (ou porta configurada no Vite)
- Backend API: http://localhost:3000

- Swagger: http://localhost:3000/docs

Variáveis de ambiente

O projeto utiliza variáveis de ambiente para configuração segura de banco de dados, autenticação e comunicação entre serviços.

Backend (.env)

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/stockdb"
JWT_SECRET=supersecret
PORT=3000

ADMIN_EMAIL=admin@system.com
ADMIN_PASSWORD=123456
ADMIN_NAME="Administrador do Sistema"
```

Frontend (.env)

```
VITE_API_URL=http://backend:3000
```

> Observação: no ambiente Docker, o frontend consome a API via nome do serviço `backend`.

Arquivo de exemplo

O projeto inclui arquivos `.env.example` para facilitar a configuração do ambiente sem expor dados sensíveis.

Uso de `useCallback` (otimização de performance)

ProductModal

O `useCallback` foi aplicado nas funções `validate` e `handleSubmit` para evitar recriação desnecessária em cada renderização do modal.

handleSubmit

```
consthandleSubmit=useCallback(() => {
if (!validate())return;
onSubmit();
}, [onSubmit,validate]);
```

Justificativa: mantém referência estável da função e evita reprocessamentos no fluxo de submit.

---

validate

```
constvalidate=useCallback(() => {
  ...
}, [form.name,form.sku,form.stock]);
```

Justificativa: evita recriação da função de validação a cada render, garantindo consistência no uso dentro do submit.

CreateUserModal

validate

```
constvalidate=useCallback(() => {
  ...
}, [form.email,form.password,isEdit]);
```

Justificativa: evita recriação da validação e garante consistência no fluxo de criação/edição.

Uso de `useMemo` (otimização de performance)

UsersTable e ProductTable

O `useMemo` foi utilizado para memoizar a definição das colunas das tabelas (`columns`), evitando recriação a cada renderização.

UsersTable

```
constcolumns=useMemo(() => [
  ...
], [onEdit,setDeleteId]);
```

ProductTable

```
constcolumns=useMemo(() => [
  ...
], [onEdit]);
```

Justificativa: evita re-renderizações desnecessárias do DataGrid ao manter a referência estável das colunas.

Benefícios

- Redução de re-renderizações desnecessárias,melhor performance em tabelas com paginação,funções e estruturas com referências estáveis,m elhor eficiência em componentes mais pesados

---

O uso de `useCallback` e `useMemo` foi aplicado de forma estratégica, apenas onde há impacto real de performance (tabelas e formulários), evitando uso excessivo.

Página de Autenticação

A aplicação possui uma página unificada para **login** e **cadastro de usuários**, com alternância dinâmica entre os formulários sem navegação de rota.

Login

Permite autenticação via email e senha, com validações básicas e tratamento de erros retornados pela API (como credenciais inválidas).

Após sucesso, o usuário é redirecionado para a aplicação e o token JWT é armazenado.

Cadastro

Permite criação de novos usuários com validações de campos obrigatórios, formato de email, telefone e confirmação de senha.

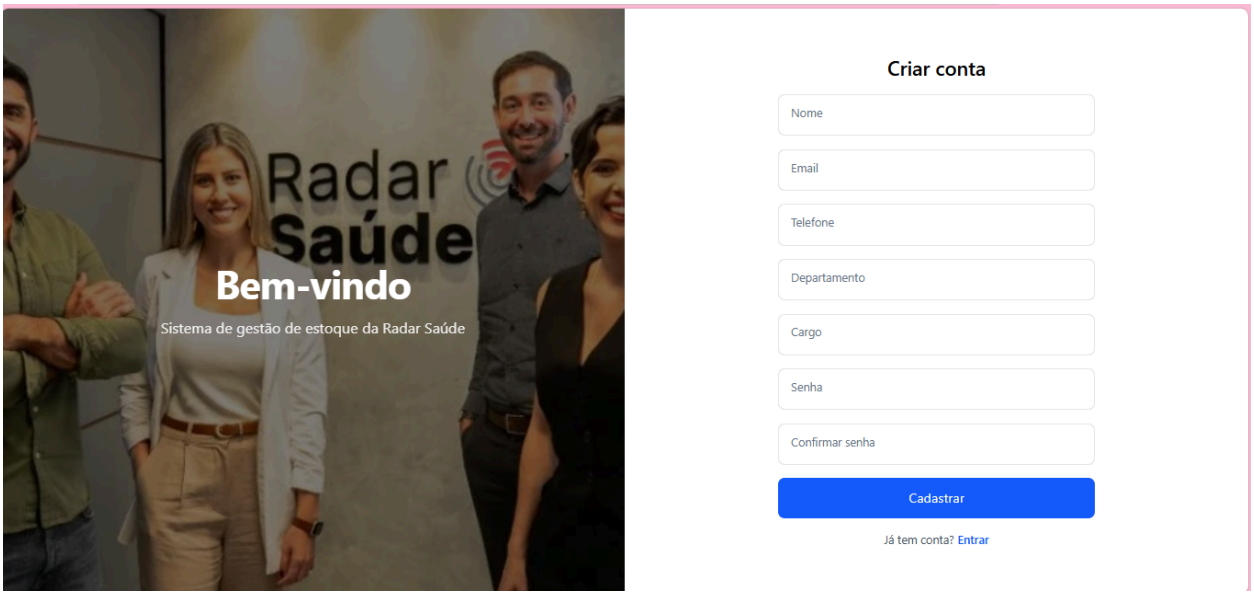
Erros como email já cadastrado são exibidos diretamente no formulário.

UX e comportamento

- Feedback visual de erros por campo
- Estados de loading durante requisições
- Botões desabilitados durante envio
- Alternância fluida entre login e cadastro

Segurança

- Autenticação baseada em JWT
- Envio automático do token via interceptor
- Logout automático em caso de sessão expirada (401)



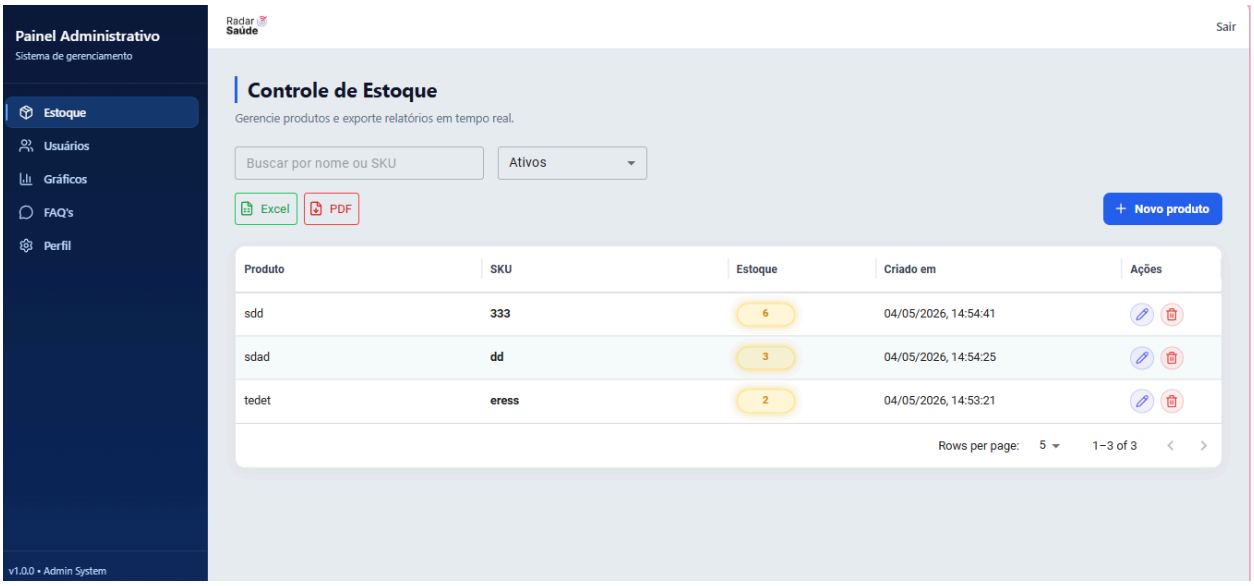
Página de Estoque de Produtos

A página de estoque permite o gerenciamento completo dos produtos cadastrados no sistema. Nela, é possível visualizar os itens em uma tabela paginada, com informações como nome, SKU, quantidade em estoque e data de criação.

O usuário pode buscar produtos por nome ou SKU e, caso tenha perfil de administrador, filtrar por status (ativos, inativos ou todos). A interface também disponibiliza ações para criar, editar e excluir produtos.

A criação e edição são realizadas através de um modal com validações de formulário, incluindo verificação de SKU duplicado. A exclusão exige confirmação para evitar ações acidentais.

Além disso, a página oferece exportação dos dados em Excel e PDF, facilitando a geração de relatórios.



Página de Gestão de Usuários

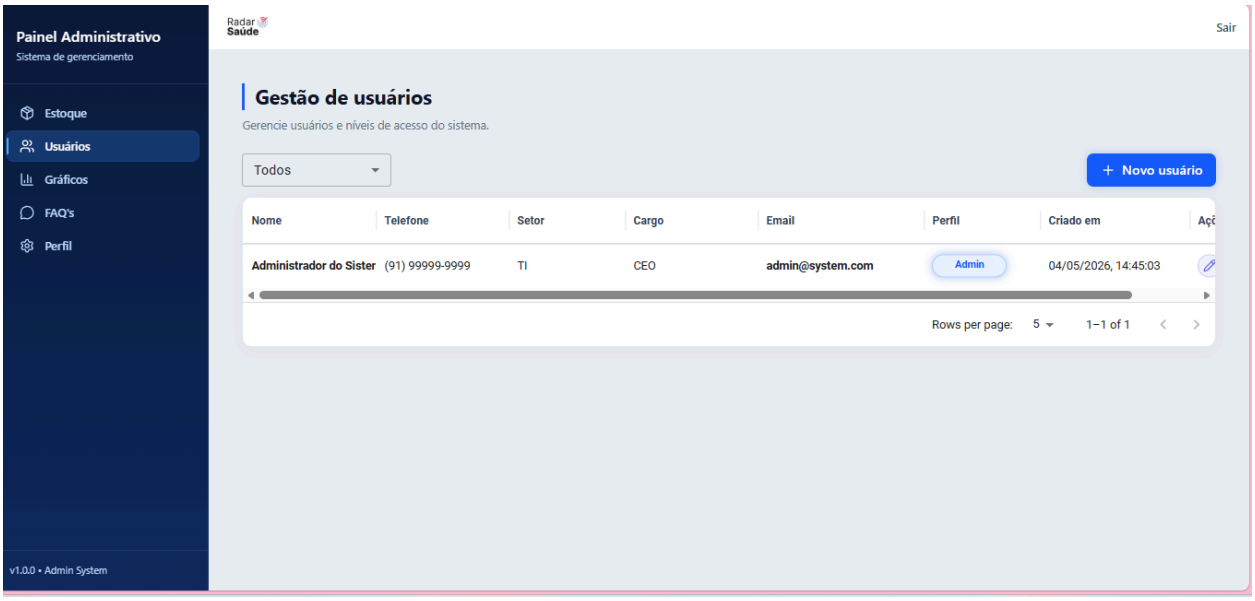
A página de usuários é **restrita a administradores** e permite o gerenciamento completo dos usuários do sistema.

Nela é possível listar usuários com paginação, visualizar informações como nome, email, telefone, cargo, setor e perfil de acesso.

O administrador pode filtrar usuários por tipo de perfil (admin ou user), além de criar, editar e excluir registros.

O cadastro e edição são realizados através de um modal com validação de campos obrigatórios e regras de negócio, como formato de email e consistência de dados. Também é possível definir o nível de acesso do usuário (admin ou user) por meio de um switch.

A tabela de usuários também permite ações rápidas de edição e exclusão com confirmação.



Página de Dashboard

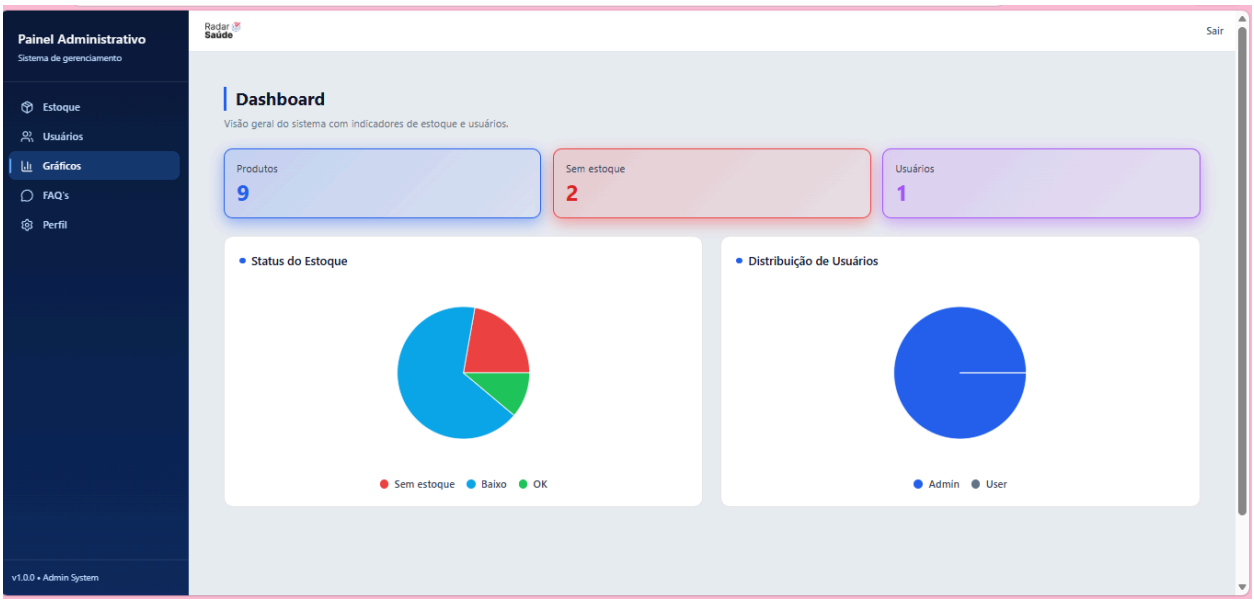
A página de dashboard apresenta uma visão geral do sistema com indicadores de estoque e usuários.

Ela exibe cards com métricas principais, como total de produtos, produtos sem estoque e total de usuários. Além disso, contém gráficos em formato de pizza que mostram a distribuição do estoque (sem estoque, baixo e ok) e a distribuição de usuários (admin e user).

Os dados são carregados dinamicamente via API e atualizados ao carregar a página. Durante o carregamento, é exibido um estado de loading.

O dashboard também inclui estados de fallback quando não há produtos cadastrados, evitando gráficos vazios e melhorando a experiência do usuário.

A interface é responsiva e utiliza animações suaves para entrada dos componentes, além de componentes reutilizáveis para cards, gráficos e legendas.



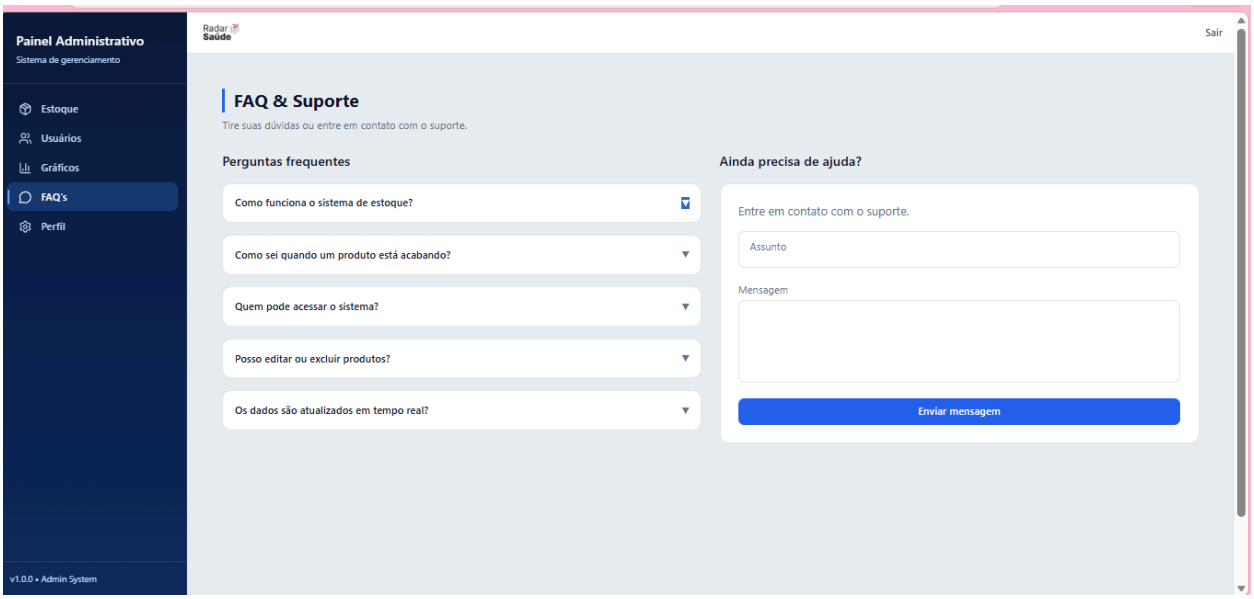
página FAQ & Suporte

A página de FAQ tem como objetivo central oferecer suporte ao usuário dentro do sistema, combinando uma área de dúvidas frequentes com um formulário de contato.

Ela é dividida em duas seções principais:

- **FAQ (Perguntas Frequentes)**: lista interativa com perguntas e respostas sobre o funcionamento do sistema (estoque, permissões, usuários e atualizações em tempo real). As perguntas podem ser expandidas e recolhidas individualmente.
- **Formulário de contato**: permite ao usuário enviar uma mensagem de suporte informando assunto e descrição. Após o envio, é exibido um modal de confirmação simulando o fluxo de atendimento (sem integração real com envio de e-mail).

A tela utiliza animações suaves para transição dos elementos e mantém um layout responsivo, dividindo FAQ e formulário lado a lado em telas maiores e em coluna em dispositivos menores.



página Perfil

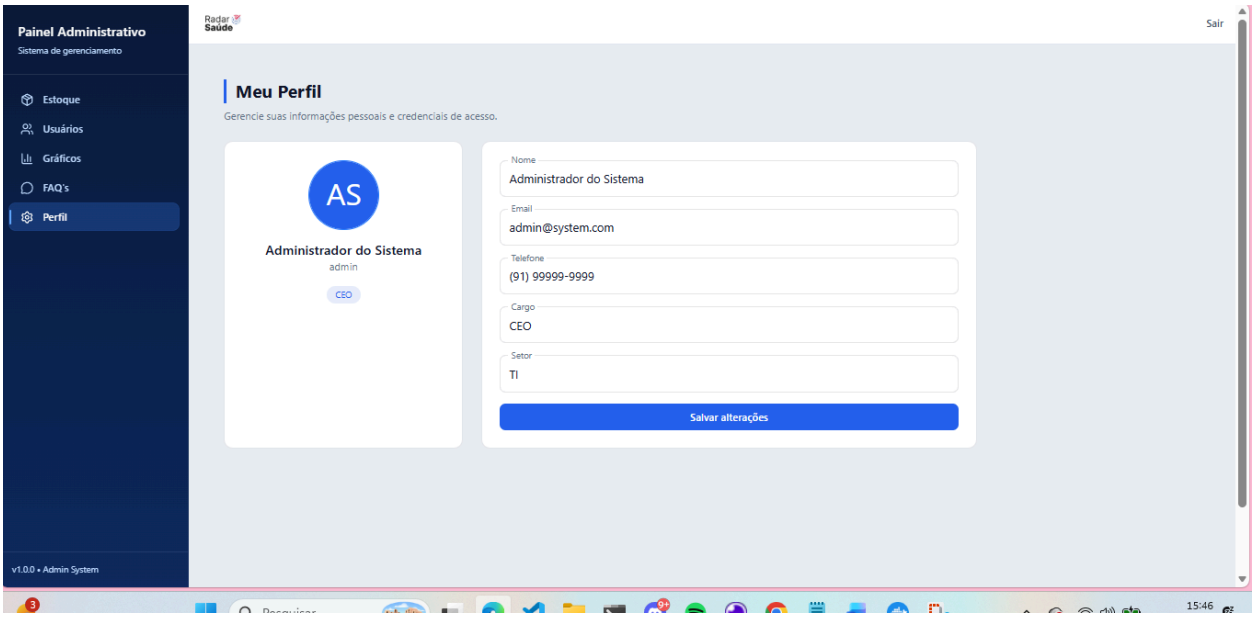
A página de perfil permite que o usuário visualize e edite suas próprias informações pessoais dentro do sistema.

Ela é composta por duas áreas principais:

- **Card de perfil**: exibe foto (ou avatar gerado automaticamente), nome, papel (admin/user) e cargo atual do usuário.
- **Formulário de edição**: permite atualizar dados como nome, e-mail, telefone, cargo e setor. O telefone possui formatação automática e os dados são validados antes do envio.

Ao salvar, as informações são enviadas para atualização no backend e uma notificação indica sucesso ou erro da operação.

A tela também realiza a busca automática dos dados do usuário logado ao ser carregada, garantindo que as informações estejam sempre atualizadas.



Exemplos de uso da API

Login

```
POST /auth/login
```

```
{
  "email":"admin@system.com",
  "password":"123456"
}
```

Resposta:

```
{
  "token":"jwt_token_aqui"
}
```

Swagger

A documentação interativa da API está disponível em:

```
http://localhost:3000/docs
```

Considerações finais

Agradeço pela oportunidade e pela confiança em avaliar este projeto.

Este desafio foi uma excelente experiência para demonstrar minhas habilidades em desenvolvimento fullstack, arquitetura de software, boas práticas de código e organização de projetos reais.

Fico à disposição para qualquer dúvida ou feedback sobre a implementação.

Tenho grande interesse em fazer parte da equipe da **Radar Saúde** e contribuir com meu crescimento técnico e com o time.

Contato

- 📱 Telefone: (85) 98233-3175
- 📧 Email: giuliatrevisan.job@gmail.com